

Data Leakage Risks in AI-Assisted Feature Engineering Pipelines

Abstract

Feature engineering is a critical stage in machine learning because appropriately constructed features can substantially improve predictive performance, interpretability, and computational efficiency. Recent advances in artificial intelligence (AI), particularly large language models (LLMs), have introduced new approaches for automating or assisting feature engineering. These systems can generate transformations, aggregate variables, select candidate features, and iteratively refine feature sets using model feedback. However, increased automation also creates additional opportunities for data leakage. Data leakage occurs when information unavailable at prediction time, or information from evaluation data, influences feature construction, feature selection, preprocessing, model selection, or evaluation. Leakage can produce overly optimistic performance estimates and models that fail when deployed on genuinely unseen data.

This paper examines the risks of data leakage in AI-assisted feature engineering pipelines. It first reviews the classical literature on leakage, preprocessing, automated feature engineering, and model-selection bias. It then analyzes how leakage can arise at different stages of AI-assisted feature engineering, including data preprocessing, feature generation, target encoding, temporal aggregation, feature selection, iterative validation, prompt construction, and automated search. Particular attention is given to the interaction between LLMs and conventional machine learning pipelines. Unlike traditional automated feature engineering, LLM-assisted systems may use natural-language descriptions, metadata, examples, validation feedback, and iterative reasoning to generate candidate features. Consequently, leakage can occur not only through numerical transformations but also through information supplied to the AI system.

The paper argues that leakage prevention should be treated as a pipeline-level property rather than a model-level property. Training-test separation, temporal validity, nested validation, provenance tracking, leakage-aware feature generation, and human oversight are therefore essential. Finally, the paper identifies research gaps concerning automated leakage detection, provenance-aware LLM feature engineering, standardized benchmarks, causal validation, and reproducibility. Addressing these challenges is necessary for trustworthy and deployable AI-assisted machine learning systems.

Keywords: data leakage, feature engineering, artificial intelligence, large language models, automated feature engineering, machine learning pipelines, data preprocessing, model validation, AutoML, trustworthy AI

1. Introduction

Machine learning systems are increasingly constructed as multi-stage pipelines rather than isolated predictive models. A typical pipeline may include data collection, cleaning, preprocessing, feature engineering, feature selection, model training, hyperparameter optimization, evaluation, and deployment. Feature engineering occupies a particularly important position because it determines how raw observations

are represented to the learning algorithm. Well-designed features can expose meaningful relationships that are difficult for a predictive model to learn directly, whereas poorly designed features can introduce noise, redundancy, bias, or information that would not be available during real-world prediction.

A fundamental threat to the validity of machine learning pipelines is **data leakage**. Kaufman et al. defined leakage as the introduction of information concerning the prediction target that should not legitimately be available to the modeling process [1]. Leakage is particularly problematic because it can remain hidden while producing apparently excellent predictive performance. The resulting model may therefore be selected, published, or deployed on the basis of an invalid estimate of generalization performance.

Data leakage is not limited to directly including the target variable as an input. It can occur indirectly through preprocessing, feature selection, aggregation, duplicated observations, temporal violations, inappropriate validation procedures, and model-selection decisions. For example, if a normalization or feature-selection operation is fitted using the complete dataset before a train-test split, information from the test set influences the representation learned by the model. The resulting evaluation no longer represents performance on genuinely unseen data. Modern machine learning guidance explicitly identifies this as a common source of leakage and recommends fitting transformations only on training data.

The emergence of AI-assisted feature engineering adds another dimension to this problem. Traditional automated feature engineering systems generate candidate features through predefined mathematical operators, relational aggregations, or search procedures. Examples include Deep Feature Synthesis and ExploreKit [4,5]. More recently, LLM-based systems have been developed to generate semantically meaningful features using natural-language descriptions of datasets and domain knowledge [6]. CAAFE, for example, uses an LLM to generate additional features for tabular data based on contextual information about the dataset.

LLMs may also participate in iterative feature search, where generated features are evaluated, feedback is returned to the LLM, and subsequent features are proposed. Recent work such as LLM-FE frames automated feature engineering as a program-search problem in which LLMs propose feature-transformation programs and use data-driven feedback to refine the search [7].

Although these developments can reduce manual effort and incorporate domain knowledge, they create new leakage pathways. An AI system may inadvertently receive information about validation outcomes, future observations, target-derived statistics, or metadata that would not be available at deployment time. Furthermore, repeated experimentation can cause a validation set to become an implicit training resource. Thus, the central research question is not simply whether an individual generated feature is valid, but whether the **entire AI-assisted feature engineering process preserves the separation between information available during development and information available during deployment**.

This paper therefore investigates the following questions:

1. What forms of data leakage commonly occur in feature engineering pipelines?
2. How can AI-assisted feature engineering introduce or amplify these risks?
3. What technical mechanisms can prevent leakage during automated feature generation and selection?
4. What research gaps remain in developing leakage-resistant AI-assisted pipelines?

2. Background and Related Work

2.1 Data Leakage in Machine Learning

The classical concept of data leakage predates current generative AI systems. Kaufman et al. described leakage as one of the major mistakes in data mining and emphasized the importance of separating the learning process from the prediction process [1]. Their framework shows that leakage can be subtle because it may be introduced during data collection, aggregation, preparation, or modeling rather than through an obvious target variable.

More recent research has expanded this perspective. Sasse et al. provide a comprehensive overview of leakage scenarios in supervised machine learning and emphasize that leakage may occur during preprocessing, feature selection, model selection, validation, and handling of dependent observations [2]. They argue that any data-driven decision in a pipeline should ultimately be evaluated using genuinely unseen information.

This broad definition is important for AI-assisted feature engineering. Feature engineering itself contains many data-driven decisions: which variables should be combined, which transformations should be applied, which generated features should be retained, and which feature-generation strategy should be explored. If these decisions are influenced by test data, leakage can occur even when the final model never directly observes the test labels.

2.2 Preprocessing Leakage

Preprocessing is often regarded as a harmless preparatory step, but many preprocessing algorithms learn parameters from data. Examples include standardization, normalization, imputation, dimensionality reduction, and feature selection. If these parameters are estimated using both training and test observations, the test data indirectly influences the model.

The scikit-learn documentation illustrates this problem using feature selection. Selecting features using the complete dataset before splitting can lead to performance substantially above chance even when the features and labels are statistically independent. The correct procedure is to split the data first and perform feature selection using only the training portion.

Pipeline-based approaches are therefore recommended because they allow preprocessing and modeling steps to be fitted together inside each training fold. This is especially important during cross-validation, where a transformation must be learned independently within every training fold.

2.3 Feature Engineering and Automated Feature Engineering

Feature engineering traditionally depends heavily on domain knowledge. Practitioners may construct ratios, counts, temporal indicators, interaction terms, statistical summaries, and domain-specific scores.

Automated feature engineering attempts to reduce this manual burden. Kanter and Veeramachaneni introduced Deep Feature Synthesis (DFS), which automatically constructs features by composing transformation and aggregation primitives across relational and temporal data [4]. Featuretools implements this approach and demonstrates automated feature construction over relational datasets.

ExploreKit similarly automates feature generation and selection by generating candidate features and evaluating their usefulness [5]. Katz et al. presented ExploreKit at the 2016 IEEE International Conference on Data Mining, demonstrating a systematic approach to automated feature generation and selection.

These systems demonstrate that automation can increase the scale of feature exploration. However, the more feature candidates and evaluation cycles are introduced, the more important strict separation between training and evaluation information becomes.

2.4 Model Selection and Validation Bias

Leakage is closely related to overfitting during model selection. Cawley and Talbot showed that model-selection procedures can themselves overfit when repeated decisions are made using finite validation data [8]. Similarly, Varma and Simon demonstrated that using cross-validation both to optimize a model and to estimate its final error can produce biased estimates of generalization performance [9].

Nested cross-validation addresses this issue by separating model selection from final model assessment. In nested validation, an inner loop is used for feature selection, hyperparameter tuning, and other data-dependent decisions, while an outer loop evaluates the resulting pipeline on data not used during those decisions [2,9].

This distinction becomes especially important for AI-assisted feature engineering because LLM-based systems may perform many rounds of feature generation and evaluation.

3. Main Thematic and Technical Discussion

3.1 The AI-Assisted Feature Engineering Pipeline

An AI-assisted feature engineering pipeline can be represented conceptually as:

Raw data → preprocessing → AI feature generation → feature evaluation → feature selection → model training → validation → deployment

The AI component may receive:

- column names and descriptions;
- data types;
- domain information;
- sample records;
- correlations;
- model performance;
- feature importance;
- previous feature-generation attempts;
- validation results; and
- human feedback.

The central security and validity problem is that not all of these information sources are necessarily available at prediction time.

A feature engineering pipeline is leakage-safe only if every information source used to create, evaluate, or select features would legitimately be available under the intended deployment scenario.

This leads to an important principle:

Feature availability must be defined according to prediction time, not according to whether a variable exists somewhere in the dataset.

For example, a patient's discharge diagnosis may exist in a hospital database, but if the prediction task is to estimate risk at admission, that diagnosis is future information. Using it to construct a feature would constitute target or temporal leakage.

3.2 Preprocessing Leakage

Suppose a dataset contains observations X and target y . A common but incorrect procedure is:

$$X' = T(X)$$

where transformation T is fitted using all observations, followed by:

$$X'_{train}, X'_{test} = Split(X')$$

The transformation has already incorporated information from the future test set.

The correct approach is:

$$X_{train}, X_{test} = Split(X)$$

followed by:

$$T_{train} = Fit(X_{train})$$

and:

$$X'_{train} = Transform(T_{train}, X_{train})$$

$$X'_{test} = Transform(T_{train}, X_{test})$$

The distinction appears simple, but AI-generated code can make this error easier to introduce. An LLM may generate preprocessing code that calculates global statistics before performing the split. If the resulting code is accepted without pipeline-level validation, the AI has effectively automated a leakage-prone operation.

3.3 Feature Selection Leakage

Feature selection is particularly dangerous because it directly uses relationships between features and the target.

Consider a dataset with thousands of candidate features. If the entire dataset is used to calculate feature-target correlations and select the top variables, the test labels influence which features are ultimately presented to the model.

The model may never directly receive the test labels, yet information from those labels has already influenced its input representation.

The correct procedure is:

1. Split the data.
2. Perform feature selection using training data only.
3. Apply the selected transformation to validation/test data.
4. Evaluate once on genuinely unseen data.

The same principle applies to AI-assisted feature selection. If an LLM receives test-set feature importance or validation scores and uses them to decide which generated feature should survive, the evaluation set has become part of the feature-selection process.

3.4 Target Leakage

Target leakage occurs when a feature contains information directly or indirectly derived from the target.

For example, suppose a model predicts whether a customer will default on a loan. A feature such as “number of collections after default” would be highly predictive, but it is unavailable at the time the loan-risk prediction is supposed to be made.

AI systems can generate such features because the feature may appear statistically useful even though it violates the prediction-time information boundary.

The problem can be summarized as:

$$Feature = f(Data_{future})$$

If $Data_{future}$ is unavailable when the prediction is generated, the feature is invalid regardless of how accurate it appears during testing.

3.5 Temporal Leakage

Temporal datasets require special consideration. Randomly dividing observations can allow future information to influence predictions about the past.

For a forecasting problem, the valid relationship is:

$$Past \rightarrow Prediction_{future}$$

whereas random cross-validation may create:

$$Future \rightarrow Prediction_{past}$$

This violates the actual deployment setting.

Sasse et al. identify temporal splitting as an important leakage consideration because ordinary random cross-validation can disrupt the temporal ordering of observations.

AI-assisted feature engineering can amplify this risk through automatically generated rolling or aggregation features. For example, an LLM might generate a feature such as “average customer spending over the next 30 days.” Such a feature can appear highly predictive but is invalid if the prediction is made at the beginning of that period.

Therefore, every temporal feature should be associated with an explicit **cutoff timestamp**.

3.6 Aggregation Leakage

Aggregation is common in automated feature engineering. A system may generate:

- total purchases;
- average transaction value;
- number of previous visits;
- maximum historical measurement;
- average activity during a period.

These features are valid only if the aggregation window ends before prediction time.

For example:

$$HistoricalAverage_i(t) = \frac{1}{N} \sum_{j:t_j < t} x_{ij}$$

is valid when predicting at time t .

In contrast:

$$Average_i = \frac{1}{N} \sum_{j=1}^N x_{ij}$$

may include observations occurring after t , creating temporal leakage.

Automated feature engineering systems that operate on complete relational datasets must therefore understand not only relationships between tables but also the temporal semantics of those relationships.

3.7 Target Encoding and Statistical Encoding

Categorical feature encoding can also leak target information. Target encoding replaces a category with a statistic based on target outcomes, such as the mean target value for that category.

For example:

$$TE(c) = E[Y|C = c]$$

If this statistic is calculated using the complete dataset, validation observations influence their own encoded representation.

AI-assisted systems may recommend target encoding because it can improve performance, but the implementation must use training-only statistics. Cross-fitting can further reduce leakage by ensuring that each training observation is encoded using statistics calculated without that observation.

3.8 Iterative AI Feature Search and Validation Leakage

Traditional feature engineering often involves a limited number of human decisions. AI systems can conduct thousands of candidate transformations rapidly.

Suppose an LLM generates 1,000 features and each feature is evaluated using the same validation set. The feature with the highest validation score is selected.

Even if the LLM never receives the test labels, the validation set has been repeatedly used to guide feature discovery. It is therefore no longer an unbiased estimate of generalization.

The problem resembles repeated model selection. Cawley and Talbot demonstrated that repeated selection based on finite validation data can cause selection bias [8]. Varma and Simon similarly showed that model optimization and error estimation should not be performed using the same validation information [9].

AI-assisted feature engineering therefore requires a distinction between:

- **search data;**
- **validation data;**
- **model-selection data;** and
- **final test data.**

The final test set should remain untouched until the complete feature-generation and model-selection procedure has been finalized.

3.9 Leakage Through LLM Context

LLM-assisted feature engineering introduces a leakage mechanism that is less prominent in traditional AutoML.

An LLM may receive a prompt containing:

- dataset descriptions;
- feature statistics;
- target distributions;
- validation performance;
- examples of successful features;
- model errors;
- feature importance rankings.

Suppose the system receives a feature-performance table calculated using the complete dataset. The LLM can use this information to generate new features. Even if the generated feature itself looks legitimate, its creation was influenced by information from the evaluation data.

Therefore, the **prompt is part of the machine learning pipeline** and must be treated as a potential leakage boundary.

3.10 Feature-Engineering Search as an Optimization Problem

Recent LLM-based systems treat feature engineering as a search or optimization problem. CAAFE uses LLMs to generate semantically meaningful features from dataset context [6].

LLM-FE extends this idea by using evolutionary search and iterative feedback to discover feature transformations [7].

Human-LLM collaborative approaches have also been proposed, where LLMs generate candidate operations while humans contribute preferences to guide selection [10]. This illustrates an important trend: AI-assisted feature engineering is becoming increasingly iterative and interactive rather than a single preprocessing step.

As the number of iterations increases, the possibility of validation overfitting also increases. Consequently, leakage control must encompass the entire search history rather than only the final selected feature set.

4. Current Approaches and Developments

4.1 Pipeline-Based Leakage Prevention

A widely recommended solution is to place preprocessing and feature-selection operations inside a formal machine learning pipeline. The pipeline ensures that transformations are fitted only on the relevant training partition.

Scikit-learn specifically recommends pipelines as a mechanism for reducing leakage during preprocessing, feature selection, cross-validation, and hyperparameter tuning.

For AI-assisted systems, the same principle can be extended:

Data split → AI feature generation within training fold → feature selection → model → validation

The AI feature-generation process itself must be inside the appropriate validation boundary.

4.2 Nested Cross-Validation

Nested cross-validation provides a stronger solution when extensive feature selection and model tuning are performed.

The inner loop handles:

- feature generation;
- feature selection;
- hyperparameter optimization;
- model selection.

The outer loop estimates final generalization performance.

This is particularly useful for AI-assisted feature engineering because an LLM may perform multiple rounds of data-driven search. Sasse et al. explicitly recommend nested validation when model selection and model assessment must be separated.

4.3 Temporal and Group-Aware Validation

Random splitting is insufficient when observations are related by time, subject, customer, patient, device, household, or other grouping variables.

For example, if multiple records belong to the same person, records from the same person should not automatically appear in both training and test sets. Otherwise, the model can exploit subject-specific characteristics.

Sasse et al. discuss leakage arising from related observations and temporal dependencies, demonstrating that ordinary random cross-validation can produce overly optimistic performance.

AI-assisted pipelines should therefore support:

- chronological splitting;
- group-based splitting;
- entity-aware splitting;
- blocked time-series validation; and
- future holdout evaluation.

4.4 Feature Provenance

Feature provenance records how each feature was created.

For an AI-generated feature, provenance should include:

- source columns;
- transformation operations;
- timestamps;
- training fold;
- prompt version;
- LLM/model version;
- generated code;
- evaluation data used;
- feature-selection decisions.

A provenance graph can be represented as:

$$SourceData \rightarrow Transformation \rightarrow GeneratedFeature \rightarrow Evaluation \rightarrow Selection$$

Such a graph makes it possible to determine whether a feature indirectly depends on prohibited information.

4.5 Human Oversight

Human-LLM collaboration is another emerging direction. Human experts can evaluate whether generated features make domain and temporal sense, rather than relying solely on predictive performance.

Recent work on human-LLM collaborative feature engineering argues that LLMs can generate candidate transformations while human feedback helps guide selection [10].

Human review is especially important for high-stakes applications because a feature can be statistically predictive but operationally invalid.

4.6 Leakage-Aware Evaluation Frameworks

Recent research has begun systematically categorizing leakage scenarios. Sasse et al. provide a broad framework covering test-to-train leakage, preprocessing leakage, dependent observations, temporal leakage, and model-selection leakage [2].

Research specifically examining feature-selection leakage has demonstrated that leakage can significantly inflate predictive performance. For example, studies in neuroimaging have found that feature selection and repeated subjects can substantially distort performance estimates and even affect model interpretations.

These findings reinforce the need to evaluate the complete pipeline rather than focusing solely on model architecture.

5. Research Challenges and Limitations

5.1 Automated Leakage Detection

One of the most significant challenges is detecting leakage automatically.

A simple rule-based detector may identify obvious problems such as:

- target column used as a feature;
- future timestamp used as an input;
- duplicate identifiers;
- test data used during training.

However, subtle leakage may be hidden inside complex transformations.

For example:

```
Feature A + Feature B → Feature C
Feature C → normalization
normalization → feature selection
feature selection → model
```

If Feature A contains future information, the leakage propagates through the entire pipeline.

Automated systems must therefore analyze feature dependencies rather than only feature names.

5.2 LLM Hallucination and Invalid Transformations

LLMs can generate code that is syntactically correct but statistically inappropriate. An LLM may also misunderstand the prediction-time availability of a variable.

Consequently, code correctness does not imply methodological correctness.

A generated transformation can satisfy programming requirements while violating:

- temporal constraints;
- causal assumptions;
- train-test separation;
- privacy requirements;
- domain rules.

This creates a need for validation mechanisms beyond conventional software testing.

5.3 Reproducibility

LLM-generated feature engineering can be nondeterministic. Different prompts, models, model versions, temperatures, or contextual information can produce different features.

This creates reproducibility challenges.

A reproducible AI-assisted pipeline should preserve:

1. dataset version;
2. train/test partitions;
3. preprocessing configuration;
4. prompt;
5. LLM version;
6. random seed where applicable;
7. generated feature code;
8. evaluation metrics; and
9. feature-selection history.

Without these artifacts, reproducing an experiment can be difficult.

5.4 Computational Cost

Automated feature generation can produce a very large number of candidates. Evaluating each candidate through cross-validation can become computationally expensive.

Nested cross-validation is statistically attractive but increases computational requirements because feature generation and model training may be repeated for multiple folds.

Thus, leakage-safe evaluation involves a trade-off between computational cost and statistical validity.

5.5 Domain Dependence

A feature that is valid in one domain may be leakage-prone in another.

For example, an aggregate over all customer transactions may be valid for a retrospective analysis but invalid for real-time prediction.

Therefore, general-purpose AI systems cannot determine feature validity solely from mathematical operations. They require knowledge about:

- prediction time;
- intervention time;
- data collection processes;
- business rules;
- causal relationships;
- deployment workflow.

5.6 Hidden Subgroups and Spurious Correlations

Even leakage-free pipelines can fail because of hidden stratification and spurious correlations. Oakden-Rayner et al. demonstrated that machine learning systems can perform very differently across clinically meaningful subgroups even when aggregate performance appears strong [11]. Their study reported substantial differences in performance across hidden subsets and emphasized the importance of subgroup evaluation.

This issue is distinct from classical leakage but closely related to trustworthy evaluation. A leakage-free model is not automatically a reliable model.

6. Research Gaps and Future Directions

6.1 Provenance-Aware LLM Feature Engineering

Future AI-assisted feature engineering systems should treat provenance as a first-class object.

Every generated feature should have a machine-readable record describing its dependency graph. The system should be able to answer:

“Which training observations, statistics, prompts, and evaluation results influenced this feature?”

This would allow automated leakage auditing before model evaluation.

6.2 Causal Feature Validity

Most current feature engineering systems optimize predictive performance. However, predictive usefulness does not guarantee causal or deployment validity.

Future systems should incorporate causal graphs and temporal knowledge into feature generation. A candidate feature should be evaluated according to whether its information would be legitimately available under the intended intervention and prediction scenario.

This extends the classical causal perspective on leakage proposed by Kaufman et al. [1].

6.3 Leakage-Aware LLM Prompts

Prompt design should explicitly encode information boundaries.

For example, an AI system could receive:

Prediction time: 2026-01-01 00:00
Allowed information: data observed before prediction time
Forbidden information: future observations and target-derived statistics

The LLM could then generate features under these constraints.

Future research should investigate whether explicit temporal and provenance constraints can reliably reduce leakage in LLM-generated feature engineering.

6.4 Leakage Benchmarks

The research community would benefit from standardized leakage benchmarks containing controlled examples of:

- preprocessing leakage;
- temporal leakage;
- target leakage;
- duplicate leakage;
- group leakage;
- target encoding leakage;
- feature-selection leakage;
- validation feedback leakage;
- LLM prompt leakage.

A benchmark could compare AI-assisted systems using both predictive performance and leakage rate.

6.5 Leakage Detection as a Separate Model

Another promising direction is to construct an automated **leakage detector** that analyzes machine learning pipelines.

The detector could inspect:

- source datasets;
- feature dependencies;
- timestamps;
- preprocessing operations;
- prompts;
- model-selection history;
- validation procedures.

It could then assign each feature a leakage-risk score:

$$Risk(f) = P(\text{feature } f \text{ depends on unavailable information})$$

Such a system could function as an automated quality-control layer.

6.6 Human-AI Verification

A future trustworthy pipeline should not require humans to manually inspect thousands of generated features. Instead, AI could generate features while another verification system checks them.

This suggests a two-agent architecture:

Generation Agent → Leakage Verification Agent → Human Approval → Model Training

The generation agent optimizes predictive utility, while the verification agent optimizes methodological validity.

6.7 Standardized Reporting

Research papers using AI-assisted feature engineering should report:

- exact data split;
- temporal cutoff;
- feature-generation procedure;
- prompt;
- LLM model/version;
- feature-selection strategy;
- validation procedure;
- leakage checks;
- final test-set access policy.

Such reporting would improve reproducibility and make methodological errors easier to identify.

7. Conclusion

AI-assisted feature engineering has the potential to substantially reduce the manual effort required to develop machine learning models. Traditional automated feature engineering methods already demonstrate that large numbers of candidate transformations can be generated and evaluated systematically. Recent LLM-based approaches extend this capability by incorporating natural-language reasoning, domain knowledge, iterative search, and human feedback.

However, greater automation also increases the complexity of the data-leakage problem. Leakage can arise during preprocessing, feature generation, target encoding, aggregation, feature selection, validation, model selection, and even through the information supplied to an LLM prompt. Iterative AI feature search creates an additional concern because repeated use of validation results can gradually turn evaluation data into an implicit training resource.

The most important principle is therefore that leakage prevention must be implemented at the **pipeline level**. A feature is not valid simply because the final model did not directly observe the test set. Every data-dependent operation that influenced the feature must respect the same information boundary.

Leakage-resistant AI-assisted pipelines should combine training-only preprocessing, nested validation, temporal and group-aware splitting, feature provenance, explicit prediction-time constraints, leakage-aware prompts, and human oversight. Future research should focus on automated leakage detection, causal feature validation, standardized benchmarks, and provenance-aware LLM systems.

Ultimately, the objective of AI-assisted feature engineering should not be to maximize validation performance alone. The objective should be to discover features that are **predictively useful, available at prediction time, reproducible, interpretable, and valid under real-world deployment conditions**. Only under these conditions can automated and LLM-assisted feature engineering contribute reliably to trustworthy machine learning.

References

- [1] Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). *Leakage in data mining: Formulation, detection, and avoidance*. ACM Transactions on Knowledge Discovery from Data, 6(4), Article 15, 1–21. DOI: **10.1145/2382577.2382579**.
- [2] Sasse, L., Nicolaisen-Sobesky, E., Dukart, J., Eickhoff, S. B., Götz, M., Hamdan, S., Komeyer, V., Kulkarni, A., Lahnakoski, J. M., Love, B. C., Raimondo, F., Patil, K. R., et al. (2025). *Overview of leakage scenarios in supervised machine learning*. Journal of Big Data, 12, Article 135. DOI: **10.1186/s40537-025-01193-8**.
- [3] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). *Scikit-learn: Machine learning in Python*. Journal of Machine Learning Research, 12, 2825–2830. Persistent identifier: **JMLR 12:2825–2830**.
- [4] Kanter, J. M., & Veeramachaneni, K. (2015). *Deep feature synthesis: Towards automating data science endeavors*. Proceedings of the 2015 IEEE International Conference on Data Science and Advanced Analytics (DSAA), Paris, France, 1–10. IEEE.
- [5] Katz, G., Shin, E. C. R., & Song, D. (2016). *ExploreKit: Automatic feature generation and selection*. Proceedings of the 2016 IEEE International Conference on Data Mining (ICDM), 979–984. IEEE. DOI: **10.1109/ICDM.2016.0123**.
- [6] Hollmann, N., Müller, S., & Hutter, F. (2023). *LLMs for semi-automated data science: Introducing CAAFE for context-aware automated feature engineering*. arXiv preprint arXiv:2305.03403. DOI: **10.48550/arXiv.2305.03403**.
- [7] Abhyankar, N., Shojaee, P., & Reddy, C. K. (2026). *LLM-FE: Automated feature engineering for tabular data with LLMs as evolutionary optimizers*. Transactions on Machine Learning Research. arXiv:2503.14434. DOI: **10.48550/arXiv.2503.14434**.
- [8] Cawley, G. C., & Talbot, N. L. C. (2010). *On over-fitting in model selection and subsequent selection bias in performance evaluation*. Journal of Machine Learning Research, 11, 2079–2107.

[9] Varma, S., & Simon, R. (2006). *Bias in error estimation when using cross-validation for model selection*. BMC Bioinformatics, 7, Article 91. DOI: **10.1186/1471-2105-7-91**. PMID: **16504092**.

[10] Li, Z., Bansal, A., Li, J., He, S., Lu, Z., Zhang, M., Liu, Q., Yang, Y., Jain, S., Yin, M., & Li, Y. (2026). *Human-LLM collaborative feature engineering for tabular learning*. Proceedings of the International Conference on Learning Representations (ICLR 2026). arXiv:2601.21060. DOI: **10.48550/arXiv.2601.21060**.

[11] Oakden-Rayner, L., Dunnmon, J., Carneiro, G., & Ré, C. (2020). *Hidden stratification causes clinically meaningful failures in machine learning for medical imaging*. Proceedings of the ACM Conference on Health, Inference, and Learning, 151–159. DOI: **10.1145/3368555.3384468**. PMID: **33196064**.

[12] Lim, Y., & Yoon, S. (2025). *Multi-level diagnosis and evaluation for robust tabular feature engineering with large language models*. Findings of the Association for Computational Linguistics: EMNLP 2025, 4630–4655. DOI: **10.18653/v1/2025.findings-emnlp.249**.

[13] de Winter, C., Frasincar, F., de Peuter, B., Matsiako, V., Ido, E., & Klinkhamer, J. (2025). *Automated feature engineering for automated machine learning*. Knowledge-Based Systems, 321, Article 113671. DOI: **10.1016/j.knosys.2025.113671**.

[14] Ichwani, A., Kesuma, R. I., Setiawan, A., Wicaksono, I. E., & Hanifah, R. (2026). *Preventing data leakage in classification via integrated machine learning pipelines: Preprocessing, feature transformation, and hyperparameter tuning*. Jurnal Teknik Informatika, 7(1). DOI: **10.52436/1.jutif.2026.7.1.5490**.